README.md

RWAPSSF

Problem

1. Front-running
2. Token stuck in the contract
    i. There is one player and wait for the other player to join
    ii. Only one player input or reveal choice
3. Can play only once when deployed

Solution

Front-running

- use the commit-reveal. When player want to input their choice they need to hash their choice with salt `getBytes32(choice, salt)`

```
function getBytes32(uint choice, string memory salt) view  public returns (by
      require(choice >= 0 && choice <= 6);
      bytes32 saltByte = bytes32(abi.encodePacked(salt));
      bytes32 ChoiceByte = bytes32(abi.encodePacked(choice));
      return getSaltedHash(ChoiceByte, saltByte);
  }
```

- use output from hash as a input to `input(hashAns, idx)` because we do not want player know other player choice. Then use hash number to input commit

```
function input(bytes32 hashAns, uint idx) public  {
    ...
    commit(hashAns);
    ...
}
```

- reveal their choice by input their choice and salt to `revealChoice(choice, idx, salt)` and compare with commit to find the winner

```
function revealChoice(uint choice, uint idx, string memory salt) public {
    ...
    bytes32 saltByte = bytes32(abi.encodePacked(salt));
    bytes32 ChoiceByte = bytes32(abi.encodePacked(choice));

    revealAnswer(ChoiceByte, saltByte);
    ...
}
```

Token stuck in the contract

- use time limit to force player to input their choice and reveal their choice
- use withdraw function to withdraw token from the contract when time limit is reached
- use `timeOutWithDraw()` to withdraw

```
function timeOutWithDraw() public {
    require(numPlayer >= 1);
    require(block.timestamp > lastAction + deadlineDuration);
    address payable account0 = payable(player[0].addr);
    address payable account1 = payable(player[1].addr);
    // only one player
    if (numPlayer == 1) {
        account0.transfer(reward);
        _reset();
    }
    // two players, not input or input but not reveal
    else if (numPlayer == 2 && (numInput == 0 || (numInput == 2 && numRevea
        account0.transfer(reward / 2);
        account1.transfer(reward / 2);
        _reset();
    }
    // two players, only one input
    else if (numPlayer == 2 && numInput == 1) {
        // pay to the player who input
        if (player[0].isCommit == true) {
            account0.transfer(reward);
        }
        else {
            account1.transfer(reward);
        }
        _reset();
    }
    // two players, only one reveal
    else if (numPlayer == 2 && numInput == 2 && numReveal == 1) {
        // pay to the player who reveal
        if (player[0].choice != 7) {
            account0.transfer(reward);
        }
```

```
            else {
                account1.transfer(reward);
            }
            _reset();
        }
    }
```

Can play only once when deployed

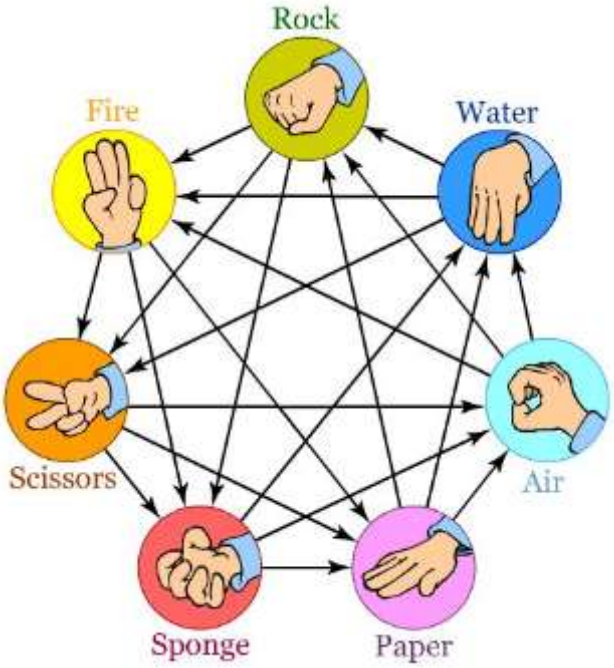
- After the game is finished, the contract will be reset and player can play again

```
    function revealChoice(uint choice, uint idx, string memory salt) public {
      ...
      if (numReveal == 2) {
          _checkWinnerAndPay();
          _reset();
      }
    }
```

```
    function _reset() private {
        numPlayer = 0;
        reward = 0;
        numInput = 0;
        numReveal = 0;
    }
```

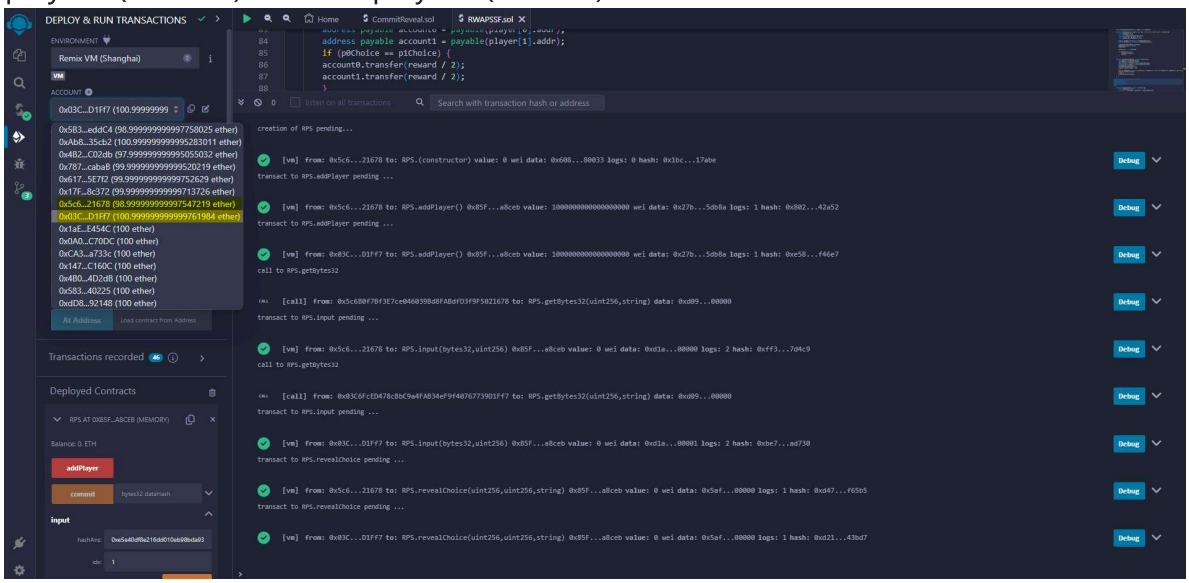
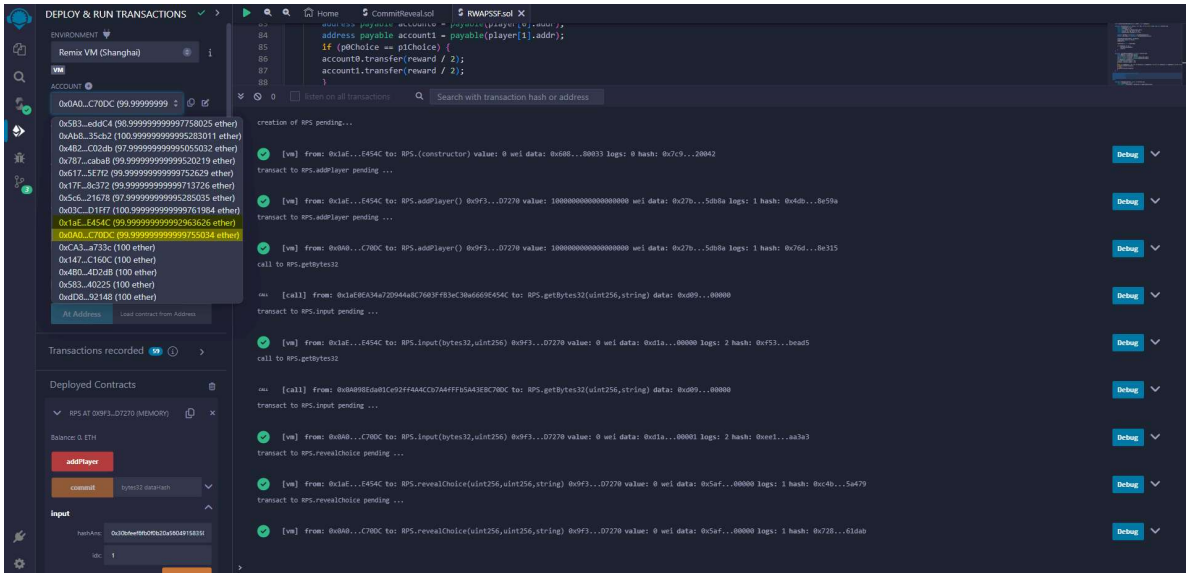
Additional

- Change Rock Paper Scissors to Rock Water Air Paper Sponge Scissors Fire



Example

- player1 (0x03C...) win and player0 (0x5c6...) lose



- draw